

Week 3 — AI-Assisted Development and Risk

CSE5IDP Industry Development Project · Presenter: Scott Mann · Online cohort notes

Semester 2, 2026 · Milestone 1 closes, Milestone 2 opens

Roughly 1h45, structured as housekeeping, a long segment on AI-assisted development, a five-minute break, then risk management. The lecture was cut slightly short because project owner meet-and-greets for Leafy AI and Tender to Bid ran immediately afterwards. The tail of the recording is face-to-face cross-talk and is largely unintelligible.

Housekeeping

Trade show — Thursday 6 August, 11:00–13:00 and 19:00–21:00.

- Use a **non-La Trobe email address** for Zep.us. The La Trobe mail server blocks or spam-flags return messages from the platform, so invitations may never reach you.
- **Only one person per team edits the booth.** Email Scott to nominate that person. With around 350 students across cohorts, simultaneous editing is unworkable. Setup takes about 15 minutes.
- He is **deliberately not prescriptive** about digital assets — PDFs, images, PowerPoints, videos, links to third-party sites are all acceptable. He wants creativity, and said the absence of a detailed must-have list is intentional.
- The Semester 1 trade show has been put on the LMS purely for idea generation.
- The booth must be staffed across both windows, but not by everyone simultaneously.

Grading of the trade show. Two rubric rows: booth materials, and communication with guests. On the second row he was specific — what is being graded is whether the visitor's question was actually answered, and whether the answer was pitched at the right technical level for that visitor. Some guests will be technical, some will not.

Two further points he stressed:

- It is a **group assessment**, so everyone contributes to building the materials and everyone contributes to answering questions. The best response to a guest question is *one-to-many* — several team members contributing to a full picture — not a one-to-one chat.
- To get full marks on the communication row you must present complex ideas **using visual tools**. Answering off the top of your head with no supporting material on screen caps your grade. Have source material in front of you when you respond.

Recording: put a Zoom link on your booth, press record when a guest joins, and use screen share. Submission is one zipped file of booth materials and one zipped file of interactions, uploaded by a single team member.

Project owner meet-and-greets are being recorded and uploaded to the LMS, so you can go back and re-analyse the owner's recommendations.

Where we are in the subject

Milestone 1 is now largely complete. Milestone 2 opens today and covers **risk management, architectural design, and teamwork**.

His recurring framing: *the hardest part of a capstone project is the people* — teammates, project owners, and

occasionally the academics. That is deliberately reflective of industry.

He also set an expectation that is worth holding on to: **no team should have a full handle on its requirements at this point**. Requirements unfold organically across the semester under agile. Anyone who thinks they have complete requirements in week 3 has probably reverted to waterfall.

One direct answer to a common email: questions like “should we use PostgreSQL or MySQL?” are not his to answer. Ask the project owner first. If the owner has no preference, it is your team’s call — do a compare and contrast, pick one, and **justify it**.

Main topic — AI-assisted development

Framed as bonus material, but it took the majority of the session. His stated position is that he is not the source of all knowledge here and wants to learn from what students are doing.

The governing principle

AI is an assistant, it is not a replacement.

The working pattern he wants is **compare and contrast**: do the human process as a team first, then prompt, then examine where the two overlap and where they diverge. Divergence is a signal to go and ask the project owner.

A related line worth remembering: *AI does not come up with ideas, it comes up with conclusions. The spark still comes from us.*

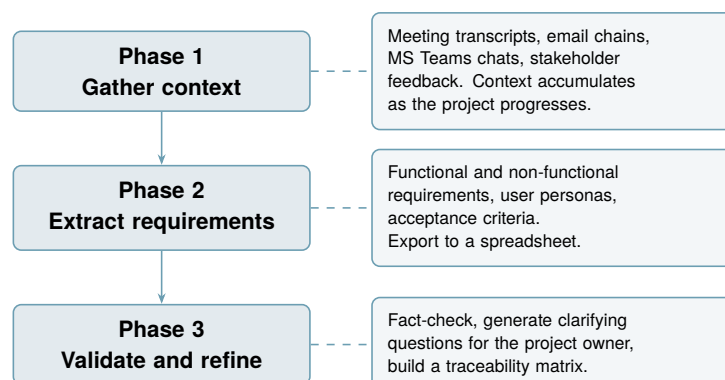
He also made an economic argument to the room. You have paid a great deal for a degree that taught you development from first principles — do not throw that investment away by outsourcing the thinking. “You learnt it, you’re gonna execute it.”

Why he thinks you cannot opt out

- Around **92 per cent of developers** were already using AI tooling in their day jobs as at the end of 2024, and he expects that figure to be close to 100 per cent now.
- Only one hand went up in a room of 15–20 when he asked who had used an in-IDE coding assistant. He said he expects every hand up by the end of semester.
- Directed at cyber students specifically: you cannot evaluate systems you cannot read. Operating at the top level and making assumptions about what is happening underneath is a flawed position. His analogy — a mechanic does not diagnose a car from the driver’s seat, they pop the hood.
- The value propositions he named: reduction of repetitive coding, **faster feature release to market**, and a reported **60 per cent improvement in code quality metrics** when test automation is built into a DevSecOps workflow.
- Onboarding onto an unfamiliar codebase used to take three to five weeks. With AI-assisted code comprehension he reckons that shrinks to a week or less — and employers cannot wait five weeks for you to become productive.

The three-phase requirements workflow

He was emphatic that throwing a requirements document at a model is a naive approach.



He rates AI as **particularly useful for non-functional requirements**, since those are harder to spot manually — regulatory controls, standards compliance, competitor behaviour, organisational policy.

On user personas: fictitious entities modelling the end user. If a model generates a persona that matches what the project owner would have described, that is a useful signal.

The chained demonstration

He worked a mobile-app case study end to end, using a short stakeholder quote as the only seed input, and explicitly encouraged replicating the chain:

1. **Stakeholder transcript in.** A few sentences about field technicians, paper forms getting lost, offline operation because cell coverage is spotty, sync on return to office, speed, and per-technician billing.
2. **Prompt for requirements.** “From the stakeholder interaction create a list of functional requirements, non-functional requirements, user stories with acceptance criteria, export as an Excel template.” Output was a spreadsheet with those categories decomposed.
3. **Feed the spreadsheet back in** to generate a solution architecture diagram.
4. **Layer the tech stack** on top. His example came back with React Native, Expo, TypeScript, SQLite with SQLCipher encryption, and AWS API Gateway with Lambda for background services.
5. **Annotate with software design patterns.** The result proposed MVVM for presentation and facades over the cloud interfaces.
6. **Then write code**, having learnt the framework interactively rather than from a textbook.

His estimate: that chain takes 30–60 minutes of prompting versus three to five weeks previously. He said teams can now reach MVP in one semester where it used to take two.

A hint dropped mid-demo: **architectural diagrams look great on a trade show booth**, and answering a visitor’s question by showing them what the system will look like generally scores well.

Specific uses he endorsed

- **Code comprehension.** GitHub Copilot in VS Code — highlight a block, use `/explain`. He recommended adding context to the prompt, such as “as a junior developer, explain in simple terms”. You can scope it from a whole project structure down to a single line range.
- **UI-to-code.** His favourite demo. A Figma PNG — no code, just an image — dropped into VS Code with “create a new screen named profile according to the UI in the attachment” produced working React Native in about 20 seconds. He noted this used to be one of the most time-consuming parts of any IDP project.
- **Unit tests.** AI is good at assertion-based unit tests because input and expected output are unambiguous. His worked example generated a Python `unittest` class covering bubble, insertion, selection and quick sort implementations.
- **Learning a new language or framework** interactively, by deconstructing working examples rather than reading an 850-page textbook.

Where he drew the line

- **Testing hierarchy.** AI returns its best results at the unit level. Value declines as you move up through integration, system, acceptance and deployment testing. Emergent properties, usability and non-functional validation still need human interpretation.
- **Security sign-off requires a human.** You cannot have AI certify a codebase against a standard and put your name on it. “You don’t want to be that person.”
- **Language coverage matters.** Python, C, C++ and Java are well represented in training data. Brand-new web frameworks and fringe languages are not, and output quality drops accordingly.
- **Training data quality propagates.** Rubbish in, rubbish out; insufficient data gives incomplete answers. He touched briefly on the risk of deliberately skewed training data producing the answers someone wants to see, then declined to go further down that path.
- **Quality is a human term.** Asking an AI to define and implement quality “doesn’t end well”.

He also noted La Trobe has signed an agreement with OpenAI, and students will get ChatGPT access from next year.

Risk management

Introduced after the break. His framing throughout: this is lived experience, not textbook project management.

Success criteria for a project

Criterion	Why it matters commercially
Delivered on time	Contracts carry financial penalties for late delivery — he cited \$10,000 per week as a realistic figure
On budget	Fixed-price contracts mean overruns come out of the developers' pay
Customer satisfaction	Poor delivery is a reputational hit for the client, and blame flows back to the development team, who then fix it at no cost
Team harmony	ICT is a small industry. Reputation follows you from job to job

On team harmony he made an extended point worth taking seriously: start the project smiling, end it smiling, and use your team of six as your primary LinkedIn network afterwards. He has known teams still in contact 10 to 15 years post-graduation.

The three risk categories

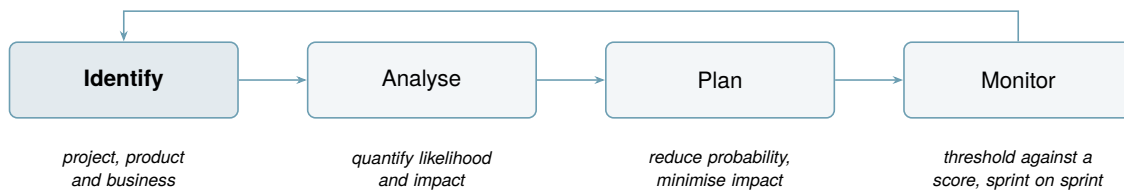
- **Project risk** — scheduling and resources. Competition for your time between subjects, commuting, and internal team scheduling. He said most teams struggle here at some point, and that this is exactly why early burndown charts look terrible and later ones look better.
- **Product risk** — quality control. Does the end result perform well enough that the owner will actually deploy it?
- **Business risk** — how the delivered product performs in market, and the reputational consequences for the project owner.

Risks he expects to see in IDP specifically

- **Students leaving mid-semester**, sometimes the day before census date (typically end of August). His mitigation: **do not let any one person be the sole expert** on any part of the project. He strongly recommended pair programming — both as knowledge-sharing insurance and because paired live coding with a senior developer is now a common job-interview format, so the practice is worth having.
- **Hardware and software unavailability**. Organisations will rarely grant students access to a production environment. Expect either a dev or pre-production mirror, or a requirement to model the technology independently on your own resources.
- **Requirements change**. Uncommon in IDP but it happens — owners see progress, get excited, and ask for more. You are allowed to push back late in semester once the must-haves and most of the nice-to-haves are delivered. Record the rest as future work.
- **Specification and contact delays**. He was candid that in the first few weeks he is inundated and may take a week to reply, and that project owners often cannot turn things around next-day either. His instruction: **do not make him the bottleneck** — find ways to stay productive and be resilient, because that is the skill that transfers to work.
- **Under-estimating task size**. He named this as one of his favourite lab exercises for the first couple of sprints, because estimates are usually inaccurate and then measurably improve. That improvement journey is what he wants to see across the semester.
- **Wrong tool, framework or stack choice**, which can block functional outcomes outright.
- **Vendor or platform migration** forcing a codebase port.
- **Product competition**. He warned about the race-to-the-bottom pattern — shipping 15 features where 11 are badly implemented, just to hold market attention.

The risk manager role

He recommends (optionally, but strongly) that **one team member takes the dedicated risk manager role**, and offered to follow it up in later lectures. The workflow:



Thresholding is the part he spent most time on. The risk manager keeps a quantitative scoreboard tracked across all five or six sprints. When an indicator trends towards a defined threshold, they raise it with the team before it materialises. The goal of good risk management is not that nothing goes wrong — it is that when something does, the impact is two days rather than three weeks.

Career angle: risk management is portfolio material. “I guided our team to a successful completion, these are the risks I managed, these are the ones that came to fruition, this is how we dealt with it.” He noted he can teach you a programming language, but not the higher-order thinking around people and strategy — and that is what employers want to hear about.

Q&A points

- **Sprint scheduling is yours to set.** Postgraduate teams get flexibility to choose their own sprint windows, provided a team of six gets through six sprints so everyone presents once. Presentations are not rigidly scheduled — if your sprint closes on a Wednesday, present that Friday, giving yourself a few days to gather outcomes from teammates rather than scrambling hours beforehand.
- **Group submissions need one copy only**, uploaded by one team member.
- **Confluence has been flaky** for some teams and has been escalated to the Atlassian admins. It is offered but not mandatory — route around it with other documentation tooling if needed.

Action items

Immediate

1. **Nominate your booth editor and email Scott**, using a non-La Trobe email address for Zep.us. Do not leave the booth build to the last moment.
2. **Build visual source material for the trade show** — architecture diagrams especially. You need something on screen when answering guest questions to score the communication row.
3. **Agree your team’s sprint windows** now that scheduling is confirmed as your call, and confirm the scrum master roster maps to six sprints.

This milestone

4. **Appoint a risk manager** and start a shared, visible risk register with a quantitative score per risk, tracked sprint on sprint.
5. **Eliminate single points of knowledge** in the team. Pair up on the database schema, the pipeline, and anything else currently owned by one person.
6. **Install GitHub Copilot in VS Code** if you have not. GitHub for Education gives students a free tier.
7. **Run the compare-and-contrast pattern** on your next sprint backlog — human process first, then prompt, then reconcile the two and take the divergences to your project owner.
8. **Check your resource assumptions** with the project owner — production access, dev or pre-production mirrors, and whether you are expected to model the environment yourself.